

Learn Rust — Full Structured Curriculum

A deliberate, compiler-led Rust curriculum focused on correctness, explicit intent, and type-driven design.

PHASE I — FOUNDATIONS (Completed)

Lesson 1 — Context, Tooling, and Rules

Lesson 2 — Variables, Bindings, and Immutability

Lesson 3 — Functions, Expressions, and Contracts

Lesson 4 — Ownership, Moves, and Borrowing

Lesson 5 — Borrow Rules and Data Flow

Lesson 6 — Enums, match, and Invalid States

PHASE II — DATA MODELLING & STRUCTURE

Lesson 7 — Structs and Data Ownership

Lesson 8 — impl Blocks, Methods, and Encapsulation

Lesson 9 — Modules, Visibility, and API Boundaries

Lesson 10 — Collections (Vec, HashMap)

Mini Project 1 — Typed CLI State Machine

Design a command-line application driven entirely by enums and match, encoding state transitions in types.

PHASE III — ERROR HANDLING & CONTROL FLOW

Lesson 11 — Option and Absence as a Type

Lesson 12 — Result and Recoverable Errors

Lesson 13 — Panic vs Failure

Lesson 14 — Iterators and Closures

Mini Project 2 — File-Based Transformer

Build a data transformation tool with explicit ownership and Result-based error handling.

PHASE IV — ABSTRACTION (CONTROLLED)

Lesson 15 — Traits as Behaviour

Lesson 16 — Generics and Type Parameters

Lesson 17 — Lifetimes (Conceptual)

Lesson 18 — Testing and Validation

PHASE V — SYSTEMS THINKING

Lesson 19 — CLI Architecture and IO Separation

Lesson 20 — Memory and Performance Intuition

MAJOR PROJECT — Rust as a Core Skill

A substantial project combining Rust with existing strengths, prioritising architectural clarity, explicit contracts, and type-driven correctness.